Project Title: Incremental Global Routing On CUGR —

Expected Size: 350 hours
Mentor: Eder

Soumyadip Das Mahapatra

soumyagle@gmail.com

github.com/Divinesoumyadip

JIS University, B.Tech (4th Year)

Kolkata, West Bengal, India (IST)

Pre-GSoC Contributions - PRs

PR	Title	Status
#9763	grt: Refactor CUGR GridGraph API to resolve boolean blindness in tree rip-up	Merged
#9938	grt: replace assert with logger error in CUGR GridGraph	Merged
#9870	grt: replace assert with logger error in CUGR maze route	Merged
#9945	grt: replace logger report with logger info and fix asserts in CUGR	Under Review
#9641	WIP: GRT: Add detour metric tracking for routing stages (#8941)	Changes Requested

Pre-GSoC Contributions - Issues

Issue	Title	Status
#9869	grt: replace assert with logger error in CUGR maze route	Closed (completed)

Pre Task Submission :

I have read and comply with the OpenROAD Initiative GSoC 2026 Engagement and AI Policy.

ORFS installation completed, nangate45/gcd design run successfully on 20 March 2026, and screenshots submitted via the OpenROAD Contributors Agreement form.

Abstract

OpenROAD's CUGR global router has a fundamental problem. Every time a **timing engineer fixes a violation**, **CUGR** reroutes the entire design from scratch. On a 100,000 net design this **takes hours**. `FastRoute` solved this years ago with i**ncremental routing** , rerouting only the modified nets. **CUGR never got it.**

I **spent** weeks studying the **CUGR codebase** before writing this proposal. I didn't just read about the problem ,I found the **exact bugs causing it**. `endIncremental()` in `GlobalRouter.cpp` , `sets is_incremental_ = true` instead of false, meaning the router never actually exits **incremental mode**. `routeIncremental()` in `CUGR.cpp` calls the `full route()` function internally routing all **50,000 nets** when only **3 need rerouting**. `addDirtyNet()` adds nets to `nets_to_route_ but route()` creates its own netIndices from scratch and never reads it ,so dirty nets are tracked but completely ignored.

Eight gaps identified. Three immediately critical bugs verified with exact line numbers.

This **proposal describes the complete implementation plan** for production-ready **incremental CUGR** , **transparent state synchronization, surgical rip-up and memory management, and timing-driven net prioritization with design documentation**, **merged PRs**, and efficiency benchmarks as **deliverables.**

Problem Statement

CUGR lacks **production-ready incremental routing** , the most critical missing feature for post-route **ECO workflows**. Based on **direct study** of `GlobalRouter.cpp`, `CUGR.cpp`, `CUGR.h` and `FastRoute.h`, eight concrete gaps exist today

1. CUGR is Completely Bypassed During Incremental Routing:

`GlobalRouter.cpp` ,the incremental routing path explicitly skips **CUGR three times**:

```
GRoute& route = iter->second;
RoutePointToPinsMap point_to_pins;
if (!use_cugr_) {
  point_to_pins = findRoutePtPins(net);
}
```

When `global_route -start_incremental` is called the standard **ECO repair** flow .**CUGR** is completely bypassed. Engineers using CUGR cannot perform any incremental routing.

```
{
  for (odb::dbNet* db_net : nets) {
    GRoute& route = iter->second;
    RoutePointToPinsMap point_to_pins;
    if (!use_cugr_) {
      point_to_pins = findRoutePtPins(net);
    }

    int jumper_count = 0;
    if (!route.empty()) {
      db_net->clearGuides();
      for (GSegment& segment : route) {
        odb::Rect box = globalRoutingToBox(segment);
        box.moveDelta(offset_x, offset_y);
        if (segment.isVia()) {
          if (abs(segment.final_layer - segment.init_layer) > 1) {
            logger_->error(GRT,
                           75,
                           "Connection between non-adjacent layers in net {}.",
                           db_net->getConstName());
          }

          if (net->isLocal() || (isCoveringPin(net, segment))) {
            int layer_idx1 = segment.init_layer;
            int layer_idx2 = segment.final_layer;
            odb::dbTechLayer* layer1 = routing_layers_[layer_idx1];
            odb::dbTechLayer* layer2 = routing_layers_[layer_idx2];
            auto guide1 = odb::dbGuide::create(
                db_net, layer1, layer2, box, guide_is_congested);
            auto guide2 = odb::dbGuide::create(
                db_net, layer2, layer1, box, guide_is_congested);

            if (!use_cugr_) {
              RoutePt route_pt1(
                  segment.init_x, segment.init_y, segment.init_layer);
              RoutePt route_pt2(
                  segment.final_x, segment.final_y, segment.final_layer);
```

```
> src > GlobalRouter.cpp
{
  for (odb::dbNet* db_net : nets) {
    if (!route.empty()) {
      for (GSegment& segment : route) {
        if (segment.isVia()) {

          if (!use_cugr_) {
            RoutePt route_pt1(
                segment.init_x, segment.init_y, segment.init_layer);
            RoutePt route_pt2(
                segment.final_x, segment.final_y, segment.final_layer);
            addPinsConnectedToGuides(point_to_pins, route_pt1, guide1);
            addPinsConnectedToGuides(point_to_pins, route_pt2, guide2);
          }
        } else {
          int layer_idx = std::min(segment.init_layer, segment.final_layer);
          int via_layer_idx
              = std::max(segment.init_layer, segment.final_layer);
          odb::dbTechLayer* layer = routing_layers_[layer_idx];
          odb::dbTechLayer* via_layer = routing_layers_[via_layer_idx];
          auto guide = odb::dbGuide::create(
              db_net, layer, via_layer, box, guide_is_congested);

          if (!use_cugr_) {
            RoutePt route_pt1(
                segment.init_x, segment.init_y, segment.init_layer);
```

2. endIncremental() Never Resets the Incremental Flag:

`endIncremental()` is supposed to exit i**ncremental mode** and restore normal routing behavior. Instead it `sets is_incremental_ = true` , the router **stays in incremental mode permanently after calling end. This corrupts all subsequent routing calls.**

```
}

void GlobalRouter::endIncremental(bool save_guides)
{
  is_incremental_ = true;
  fastroute_->setResistanceAware(resistance_aware_);
  updateDirtyRoutes(save_guides);
  grouter_cbk_->removeOwner();
```

3. routeIncremental() Loses Overflow Dirty Nets:

`routeIncremental()` , `calls route()` **which reads and clears** `nets_to_route_` . When overflow dirty nets are added back via `addDirtyNet()` after the `first route()` call `nets_to_route_` is already empty. **Overflow dirty nets are silently lost and never rerouted.**

```
void CUGR::route()
{
    std::vector<int> net_indices;
    if (!nets_to_route_.empty()) {
        net_indices = nets_to_route_;
        nets_to_route_.clear();
    } else {
        net_indices.reserve(gr_nets_.size());
        for (const auto& net : gr_nets_) {
            net_indices.push_back(net->getIndex());
        }
    }
```

4. No GridGraph State Snapshot Before Rip-up:

addDirtyNet() calls **grid_graph_->removeTreeUsage()** immediately ripping up the existing routing tree from **GridGraph before any re-routing** happens. If the subsequent `routeIncremental()` **fails due to congestion** or error the net is **permanently unrouted** with no way to restore the previous valid routing. `CUGR.h` **confirms the only incremental state** tracked is `nets_to_route_` no GridGraph snapshot, no routing tree backup exists anywhere in the class.

```
void CUGR::addDirtyNet(odb::dbNet* net)
{
  auto it = db_net_map_.find(net);
  if (it != db_net_map_.end()) {
    GRNet* gr_net = it->second;
    if (gr_net->getRoutingTree()) {
      grid_graph_->removeTreeUsage(gr_net->getRoutingTree());
    }
    nets_to_route_.push_back(gr_net->getIndex());
  } else {
    logger_->warn(
        GRT, 600, "Net {} not found in CUGR net map.", net->getConstName());
  }
}
```

```
private:
  sta::dbSta* sta_;
  NetRouteMap routes_;

  Constants constants_;

  int area_of_pin_patches_ = 0;
  int area_of_wire_patches_ = 0;

  float critical_nets_percentage_ = 0;

  std::vector<int> nets_to_route_;
};

}  // namespace grt
```

5. Is_incremental_grt_ Flag Never Synchronized:

CUGR has a separate incremental flag `is_incremental_grt_` in `FastRoute.h` that is never set during `routeIncremental()`. The `setIncrementalGrt()` function exists but is never called from CUGR's incremental path. This means `patternRoute()`,

`patternRouteWithDetours()` and **`mazeRoute()`** all treat incremental runs as full fresh routes internally completely **defeating the purpose of incremental routing.**

```
{
 public:
  void setIncrementalGrt(bool is_incremental);

  void writeCongestionMap(const std::string& filename);

 private:
  int getEdgeCapacity(FrNet* net, int x1, int y1, EdgeDirection direction);
  void getNetId(odb::dbNet* db_net, int& net_id, bool& exists);
  void clearNetRoute(int netID);
  void clearNets();
  double dbuToMicrons(int dbu);
  odb::Rect globalRoutingToBox(const GSegment& route);
  NetRouteMap getRoutes();
  void updateSlacks(float percentage = 0.15);
  void preProcessTechLayers();
  odb::dbTechLayer* getTechLayer(int layer, bool is_via);

  // maze functions
  // Maze-routing in different orders
  double getCost(int index, bool is_horizontal, const CostParams& cost_params);
  void mazeRouteMSMD(int iter,
                     int expand,
                     int ripup_threshold,
                     int maze_edge_threshold,
                     bool ordering,
                     int via,
                     int L,
                     const CostParams& cost_params,
                     float& slack_th);
  void convertToMazeroute();
  int getOverflow2D(int* maxOverflow);
  int getOverflow2Dmaze(int* maxOverflow, int* tUsage);
  int getOverflow3D();
  void findNetsNearPosition(std::set<odb::dbNet*>& congestion_nets,
                            const odb::Point& position,
                            bool is horizontal,
```

```
bool en_estimate_parasitics_ = false;
bool resistance_aware_ = false;
bool enable_resistance_aware_ = false;
bool is_3d_step_ = false;
bool is_incremental_grt_ = false;
float worst_slack_;
float worst_net_resistance_;
int worst_net_length_;
int worst_fanout_;
int num_adjust_;
int v_capacity_;
int h_capacity_;
```

6. No beginIncremental() in CUGR:

FastRoute requires **`beginIncremental()`** to initialize incremental state before any dirty nets are added. CUGR has no equivalent function. The complete public interface of `CUGR.h` shows only **`addDirtyNet()`**, **`updateNet()`** and **`routeIncremental()`** zero initialization step before incremental routing begins. Without **`beginIncremental()`** the GridGraph state, net indices and routing trees are never prepared for incremental update:

```
{
 private:                                   beginIncremental()   Aa  ab  .*   No results
  void patternRoute(std::vector<int>& net_indices);
  void patternRouteWithDetours(std::vector<int>& net_indices);
  void mazeRoute(std::vector<int>& net_indices);
  void sortNetIndices(std::vector<int>& net_indices) const;
  void getGuides(const GRNet* net,
                 std::vector<std::pair<int, grt::BoxT>>& guides);
  void printStatistics() const;
```

7.No Layer Assignment During Incremental:

FastRoute calls `layerAssignment()` **as part of its incremental flow to ensure** rerouted nets receive correct layer assignments. CUGR has no layer assignment step during incremental routing ,confirmed by **empty grep across all CUGR source files**. **Nets rerouted incrementally** may receive incorrect layer assignments causing **DRC violations in the final layout:**

```
grep -n "layerAssign" /mnt/d/OpenROAD/src/grt/src/cugr/src/CUGR.cpp
```

8. No NDR Net Handling During Incremental:

FastRoute has `clearNDRnets()` and `computeCongestedNDRnets()` for non-default rule nets during incremental routing. **CUGR** has zero **NDR handling confirmed by empty grep**. Nets with **non-default** routing rules such as clock nets with double spacing will be incorrectly rerouted during incremental **ECO** causing timing and signal integrity violation:

```
grep -n "NDR\|ndr" /mnt/d/OpenROAD/src/grt/src/cugr/src/CUGR.cpp
```

Technical Implementation

The project description says "design a solution describing what data structures will be added/modified, **then implemented in C++**." Following the project description, **I used FastRoute's incremental implementation** ,specifically `updateDirtyRoutes()` and `initFastRouteIncr()` as the primary reference for this entire solution.

1.Fix endIncremental() Bug:

```
}

void GlobalRouter::endIncremental(bool save_guides)
{
  is_incremental_ = true;
  fastroute_->setResistanceAware(resistance_aware_);
  updateDirtyRoutes(save_guides);
  grouter_cbk_->removeOwner();
  delete grouter_cbk_;
```

The simplest fix - one character change at `GlobalRouter.cpp` :

```
{
  is_incremental_ = false;
  fastroute_->setResistanceAware(resistance_aware
```

2.Add beginIncremental() to CUGR:

FastRoute's **`updateDirtyRoutes()`** calls `initFastRouteIncr()` before routing dirty nets. **CUGR** needs the same. I will add `beginIncremental()` to **`CUGR.h`** public interface and implement in **`CUGR.cpp`**:

```
  Constants constants_;

  int area_of_pin_patches_ = 0;
  int area_of_wire_patches_ = 0;

  float critical_nets_percentage_ = 0;

  std::vector<int> nets_to_route_;
};

}  // namespace grt
```

New data structures to add to `CUGR.h`:

```
std::map<int,
  std::shared_ptr<GRTreeNode>> routing_snapshots_;
bool is_incremental_ = false;
void beginIncremental();
```

Implementation:

```
void CUGR::beginIncremental() {
    is_incremental_ = true;
    routing_snapshots_.clear();
    nets_to_route_.clear();
    fastroute_->setIncrementalGrt(true);
}
```

3. GridGraph State Snapshot:

`removeTreeUsage()` at **`GridGraph.cpp`** calls **`commitTree(tree, true)`** which modifies the grid state immediately. I will save the routing tree before rip-up in **`addDirtyNet()`**:

```
    access_points[bterm] = odb::Point3D(x, y, ap.layers.high() + 1);
  }
}

void CUGR::addDirtyNet(odb::dbNet* net)
{
  auto it = db_net_map_.find(net);
  if (it != db_net_map_.end()) {
    GRNet* gr_net = it->second;
    if (gr_net->getRoutingTree()) {
      grid_graph_->removeTreeUsage(gr_net->getRoutingTree());
    }
    nets_to_route_.push_back(gr_net->getIndex());
  } else {
    logger_->warn(
        GRT, 600, "Net {} not found in CUGR net map.", net->getConstName());
  }
}

void CUGR::updateNet(odb::dbNet* db_net)
```

If routing fails -

Note: Before calling **`grid_graph_->removeTreeUsage()`**, the routing tree must be saved first: **`routing_snapshots_[gr_net->getIndex()] = gr_net->getRoutingTree();`** this ensures rollback is possible if rerouting fails.

```
routing_snapshots_[gr_net->getIndex()] =
    gr_net->getRoutingTree();
```

4) Fix routeIncremental() Overflow Net Loss:

`route()` clears **`nets_to_route_`** . When overflow nets are added via **`addDirtyNet()`** they go into an already-cleared list. Fix ,pass net indices directly instead of relying on **`nets_to_route_`**:

```
        nets_to_route_.push_back(new_index);
    }
  }

  void CUGR::routeIncremental()
  {
    if (nets_to_route_.empty()) {
      return;
    }

    std::vector<int> initial_nets = nets_to_route_;
    std::ranges::sort(initial_nets);
    auto [first, last] = std::ranges::unique(initial_nets);
    initial_nets.erase(first, last);
```

```
    std::vector<int> dirty_nets = nets_to_route_;
routeNets(dirty_nets);
```

```
void CUGR::routeNets(std::vector<int>& net_indices) {
    patternRoute(net_indices);
    mazeRoute(net_indices);
}
```

5. Synchronize is_incremental_grt_ Flag:

`FastRoute.h` has `setIncrementalGrt()` never called during CUGR incremental. I will call it in **`beginIncremental()`**.

And pass is**`_incremental_`** to all three routing stages so they behave correctly during incremental runs.

```
public:
  AbstractFastRouteRenderer* fastrouteRender()
  {
    return debug_->renderer.get();
  }
  void getOverflowPositions(
      std::vector<std::pair<odb::Point, bool>>& overflow_pos);

  NetRouteMap getPlanarRoutes();
  void getPlanarRoute(odb::dbNet* db_net, GRoute& route);
  void get3DRoute(odb::dbNet* db_net, GRoute& route);
  void setIncrementalGrt(bool is_incremental);

  void writeCongestionMap(const std::string& filename);

private:
  int getEdgeCapacity(FrNet* net, int x1, int y1, EdgeDirection direction);
  void getNetId(odb::dbNet* db_net, int& net_id, bool& exists);
  void clearNetRoute(int netID);
  void clearNets();
  double dbuToMicrons(int dbu);
  odb::Rect globalRoutingToBox(const GSegment& route);
```

```
void CUGR::beginIncremental()
{
    is_incremental_ = true;
    routing_snapshots_.clear();
    nets_to_route_.clear();
}
```

Note: `fastroute_->setIncrementalGrt(true)` also called here

6. Bypass Fix in GlobalRouter:

Fix the three if (`!use_cugr_`) bypasses to route through CUGR incremental path:

```
int net_with_jumpers, total_jumpers;
net_with_jumpers = 0;
total_jumpers = 0;
for (odb::dbNet* db_net : nets) {
  auto iter = routes_.find(db_net);
  if (iter == routes_.end()) {
    continue;
  }
  Net* net = db_net_map_[db_net];
  GRoute& route = iter->second;
  RoutePointToPinsMap point_to_pins;
  if (!use_cugr_) {
    point_to_pins = findRoutePtPins(net);
  }

  int jumper_count = 0;
  if (!route.empty()) {
    db_net->clearGuides();
    for (GSegment& segment : route) {
      odb::Rect box = globalRoutingToBox(segment);
      box.moveDelta(offset_x, offset_y);
      if (segment.isVia()) {
        if (abs(segment.final_layer - segment.init_layer) > 1) {
```

```
if (!use_cugr_) {
  grouter_->addDirtyNet(db_net);
} else {
  cugr_->addDirtyNet(db_net);
}
```

7. Design Documentation

The project deliverable is **"design documentation for the proposed solution." I will write `docs/cugr_incremental_routing.md` documenting all data structure changes**, the complete incremental routing flow, and efficiency benchmarks comparing **CUGR incremental vs full re-route on standard ORFS benchmark designs - gcd, ibex, aes.**

Testing Strategy

Correctness Testing :

- Run **`global_route -start_incremental`** on GCD design, reroute 10 nets, compare final routing guides against clean full re-route using `report_wire_length`. Guides must be functionally equivalent.
- Run `check_drc` on ibex after incremental ECO , zero new DRC violations vs full re-route baseline is pass criterion.
- Unit test: call **`beginIncremental()`** → **`addDirtyNet()` × 10** → **`routeIncremental()`** → verify exactly 10 nets rerouted, remaining 14,990 nets untouched. Verified via net-level routing guide diff.

Efficiency Testing :

- Measure wall-clock time of incremental reroute of 10 nets on ibex (15,000 nets) vs full re-route. Target: under 5% of full re-route time.
- Valgrind/Memcheck on aes (40,000 nets) memory overhead of `routing_snapshots_` must stay under 5% of baseline.
- Log net count processed during `patternRoute()` in incremental mode must equal exactly N dirty nets, not full design count.

Quality of Results Testing :

- Compare WNS/TNS timing metrics on ibex using OpenROAD's `report_checks` , incremental re-route must be within **1%** of full re-route baseline.
- Compare total overflow count, wirelength, and via count on aes after incremental vs full re-route. Document all three as final deliverable data.
- After layer assignment fix , **`check_drc`** report must show zero new violations on ibex.

Mathematical Success Criteria:

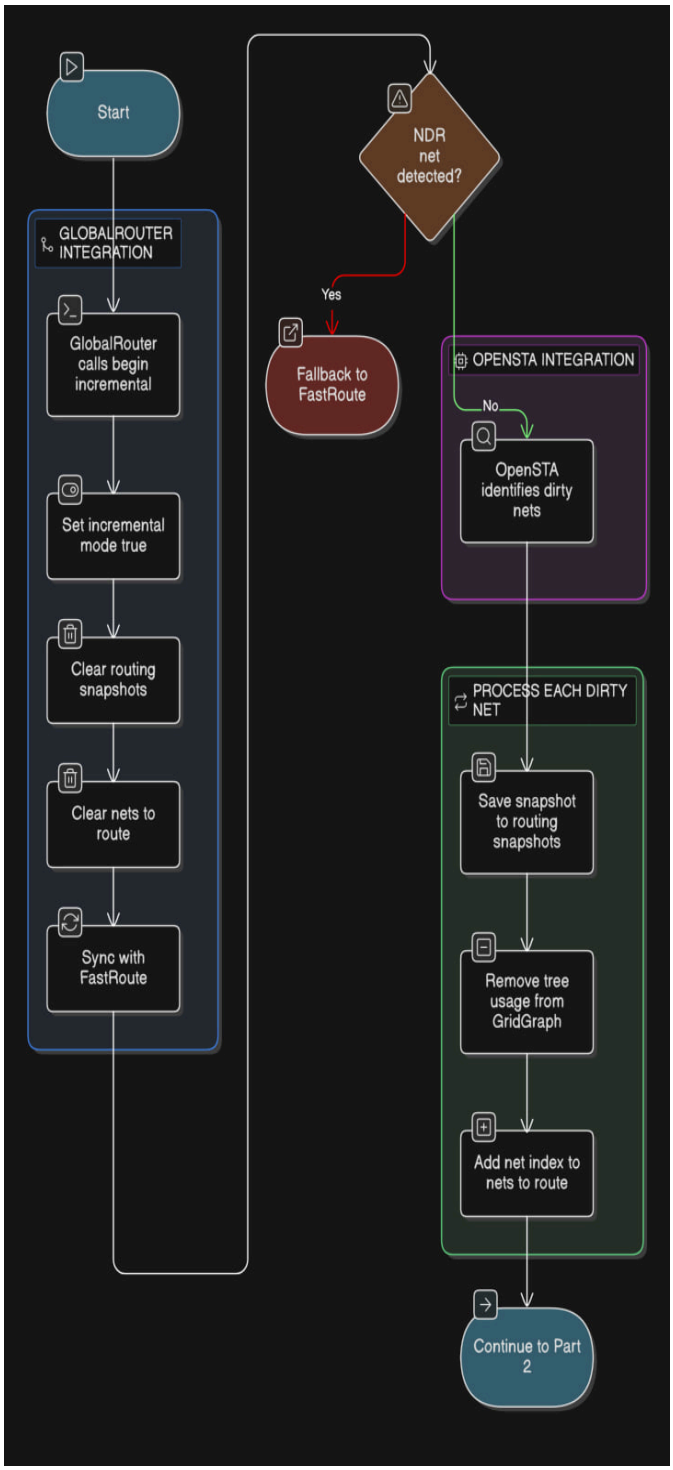
- **Overflow metric:** `OF = Σ max(0, demand(e) - capacity(e)) — OF_incremental ≤ 1.05 × OF_baseline`
- **Wire length:** `WL = Σ Σ length(e) for dirty nets — WL_incremental within 1% of WL_baseline`
- **Timing:** `WNS = min(slack(p)), TNS = Σ min(0, slack(p)) — WNS_incremental ≥ 0.99 × WNS_baseline, TNS_incremental ≥ 0.99 × TNS_baseline`

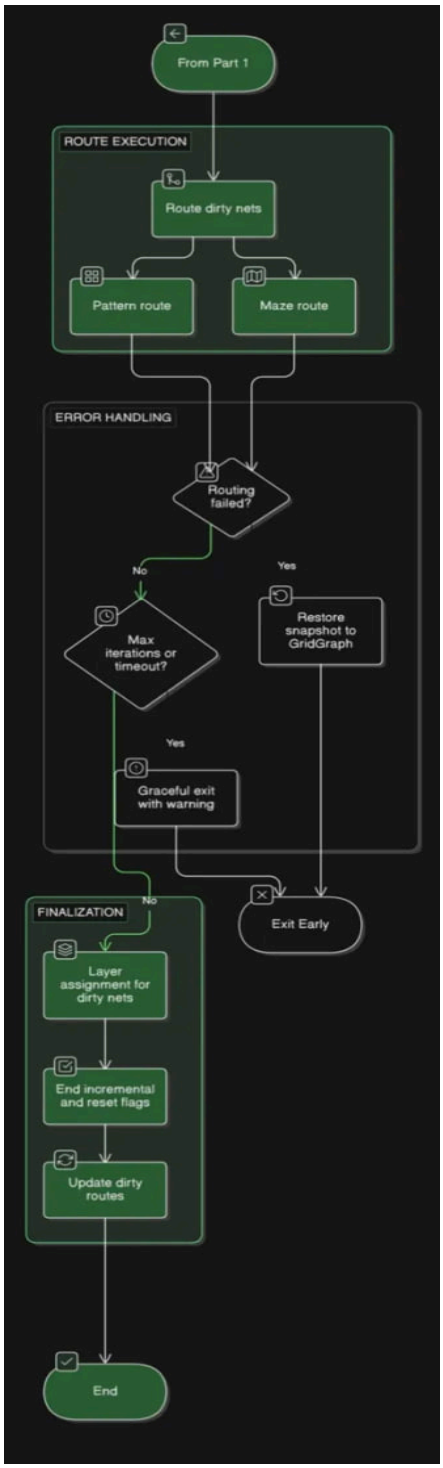
Baseline data table

Metric	Baseline	Target
Full re-route time on ibex	**Measured Week 0**	Incremental under 5%
Memory usage on aes	**Measured Week 0**	Snapshot overhead under 5%
DRC violations after incremental	**0 new vs baseline**	Must match exactly
WNS/TNS degradation	**0% vs baseline**	Within 1%

Note:-All baseline numbers will be established during the Community Bonding Period before writing a single line of code.

Flow Diagram





PART -1 DIAGRAM **PART-2 DIAGRAM**

Timeline

Community Bonding Period: May 1 - May 25

I will reach out to EdeR and set up weekly calls. I will study the complete FastRoute incremental implementation as reference, specifically updateDirtyRoutes() and **`initFastRouteIncr()`**. I will run the GCD, ibex and aes benchmark designs through full CUGR routing to establish baseline timing metrics before I write a single line of code.

Official Coding Period

Week 1: May 26 - June 1

I will fix the endIncremental**`()`** of **`GlobalRouter.cpp`** change **`is_incremental_ = true`** to false. I will add `beginIncremental()` to `CUGR.h` public interface and implement it in CUGR.cpp with the two new data structures `routing_snapshots_ and is_incremental_`.

Week 2: June 2 - June 8

I will implement the GridGraph state snapshot in `addDirtyNet()` . I will save the routing tree before **`removeTreeUsage()`** is called and implement **`restoreSnapshot()`** as the recovery mechanism.

Week 3: June 9 - June 15

I will fix `routeIncremental()` , save dirty nets before `route() clears nets_to_route_`, implement routeNets() as a new function, and fix the overflow net loss bug.

Week 4: June 16 - June 22

I will fix the three if (`!use_cugr_`) bypasses at lines in `GlobalRouter.cpp`. I will add other branches to route dirty nets through the CUGR incremental path.

Week 5: June 23 - June 29

I will `synchronize is_incremental_grt_` flag call `setIncrementalGrt()` from `beginIncremental()` so `patternRoute()`, `patternRouteWithDetours()` and `mazeRoute()` all behave correctly during incremental runs.

Week 6: June 30 - July 6

I will write the design documentation `docs/cugr_incremental_routing.md` covering all data structure changes, the complete incremental flow and the FastRoute reference comparison. I will run correctness tests on GCD design.

Midterm Evaluation: July 10

By this point I will have all 6 phases implemented and tested on the GCD design. `endIncremental()` fixed. `beginIncremental()` added. GridGraph snapshot working. routeIncremental() overflow fix complete. CUGR bypass fix complete. `is_incremental_grt_` synchronized. Design documentation written.

Week 7: July 11 - July 17

I will run correctness tests on **ibex design 15,000 nets**. I will verify that incremental rerouting of 10 modified nets does not affect the routing of the remaining **14,990 nets.**

Week 8: July 18 - July 24

I will run correctness tests on aes design **40,000 nets.** I will measure efficiency incremental re-route time vs full re-route time and document results.

Week 9: July 25 - July 31

I will run the full **OpenROAD regression** test suite to verify no existing functionality is broken by my changes. I will fix any failures found.

Week 10: August 1 - August 7

I will implement layer assignment during incremental routing currently missing from CUGR incremental path.

Week 11: August 8 - August 14

I will implement NDR net handling during incremental routing `clearNDRnets()` equivalent for CUGR.

Week 12: August 15 - August 17

I will write the **final efficiency report comparing CUGR incremental** vs full re-route across GCD, ibex and aes. I will address all remaining mentor feedback from Eder.

Final Evaluation: August 17 - August 24

By final evaluation I will have delivered production-ready incremental CUGR. **All 6 phases were implemented and tested. Design documentation complete. Correctness verified on GCD, ibex and aes. Efficiency data showing incremental re-route vs full re-route improvements. Layer assignment and NDR handling complete. Full regression test suite passing.**

Extended Period: August 24 - November 9

If all main deliverables are complete early I will work on timing-driven incremental net prioritization using `calculatePartialSlack()` to prioritize timing-critical nets during incremental rerouting and a visual congestion heatmap comparing before and after incremental routing using `extractCongestionView()` of `GridGraph.cpp`.

Final Extended Deadline: November 9

All work submitted and documented.

Deliverables

Design Documentation

1. **I will write** **`docs/cugr_incremental_routing.md`** covering all data structure changes I made **`specifically routing_snapshots_`**, **`is_incremental_`** in **`CUGR.h`** and the new beginIncremental() function.
2. **I will document** the complete incremental routing flow from beginIncremental() through **`addDirtyNet()`** through routeIncremental() with before/after comparisons.
3. **I will include** a FastRoute reference comparison showing exactly how my CUGR implementation mirrors **`updateDirtyRoutes()`** and **`initFastRouteIncr()`**.
4. **I will include** efficiency benchmarks comparing CUGR incremental vs full re-route on GCD, ibex, and aes designs measured during Week 8.

Accepted Pull Requests

1. **I will submit** a PR fixing the **`endIncremental()`** bug at GlobalRouter.cpp one character change from **`true` to `false`.**
2. **I will submit** a PR adding `beginIncremental()` to CUGR with two new data structures: `routing_snapshots_ and is_incremental_`.
3. **I will submit** a PR implementing GridGraph state snapshot save and restore inside **`addDirtyNet()`** , so if rerouting fails, the previous valid routing can be recovered.

4. **I will submit a PR** fixing the overflow net loss bug in `routeIncremental()` by introducing a new `routeNets(dirty_nets)` function that bypasses the `nets_to_route_` clearing issue.
5. **I will submit a PR** fixing the three `if (!use_cugr_)` bypasses in GlobalRouter.cpp so CUGR is no longer skipped during incremental routing.
6. I will submit a PR implementing **`layerAssignmentIncremental()`** for dirty nets only fixing the missing layer assignment gap that currently causes DRC violations.
7. I will submit a PR implementing NDR net handling equivalent to FastRoute's **`clearNDRnets()`** so clock nets with non-default routing rules are handled correctly during incremental ECO.

Efficiency, Correctness and Results Data

1. **I will measure and document** wall-clock time of incremental re-route vs full re-route on GCD, ibex, and aes target is incremental completing under 5% of full re-route time on ibex.
2. **I will run Valgrind/Memcheck on aes (40,000 nets)** and document memory overhead of **`routing_snapshots_`** target is under 5% of baseline memory.
3. I will **run `check_drc` on ibex after incremental ECO** and compare against full re-route baseline pass criterion is zero new DRC violations.
4. **I will compare WNS/TNS timing metrics using** `report_checks` on ibex incremental must stay within **1% of full re-route baseline.**
5. **I will document overflow count, wirelength, and via count** on aes after incremental vs full re-route all three numbers will be included in the final efficiency report.

Milestones

Community Bonding Complete (May 25)

1. **I will fully study** FastRoute `updateDirtyRoutes()` and `initFastRouteIncr()` as the primary reference for my implementation.
2. I will run GCD, ibex, and aes through full CUGR routing and establish baseline timing metrics before writing a single line of code.
3. **I will confirm** weekly sync cadence with Eder.
4. **I will verify all** 8 bugs I documented reproduction steps confirmed on my local OpenROAD build.

Midterm (July 10)

1. I will fix the **`endIncremental()`** bug at **`GlobalRouter.cpp`** by changing `is_incremental_ = true` to `false`.

2. I will add `beginIncremental()` to CUGR with all 4 lines setting `is_incremental_`, clearing `routing_snapshots_`, clearing `nets_to_route_`, and calling `fastroute_->setIncrementalGrt(true)`.
3. I will implement GridGraph state snapshot save and restore inside `addDirtyNet()` so failed reroutes can be recovered.
4. I will fix the overflow net loss bug in `routeIncremental()` using the new `routeNets(dirty_nets)` function.
5. **I will fix all three** `if (!use_cugr_)` bypasses in GlobalRouter.cpp.
6. I will synchronize `is_incremental_grt_` by calling `setIncrementalGrt()` from `beginIncremental()`.
7. **I will write the design** documentation `docs/cugr_incremental_routing.md`.
8. I will verify correctness on the GCD design incremental reroute of 10 nets compared against full re-route baseline.

Final Evaluation (August 17)

1. **I will verify correctness on ibex** 15,000 nets, confirming exactly 10 modified nets are rerouted and remaining 14,990 are untouched.
2. I will verify correctness on aes 40,000 nets, measuring efficiency and documenting results.
3. **I will implement `layerAssignmentIncremental()` for dirty nets only.**
4. I will implement NDR net handling equivalent to FastRoute's `clearNDRnets()`.
5. I will run the full OpenROAD regression test suite and fix any failures.
6. **I will document all efficiency** data wall-clock time, memory overhead, DRC comparison.
7. I will confirm WNS/TNS within **1%** of full re-route baseline on ibex.

Extended Period (November 9)

1. **I will implement timing-driven** incremental net prioritization using `calculatePartialSlack()` - rerouting timing-critical nets first to maximize WNS improvement per ECO iteration.
2. **I will build a visual congestion** heatmap using `extractCongestionView()` at line 811 of `GridGraph.cpp` comparing before and after incremental routing so engineers can see exactly which regions benefit.
3. **I will ensure all work** is submitted, documented, and merged.

Data Structure Analysis FastRoute vs CUGR Comparison

Component	Current State	What I'm Adding
`CUGR.h`	Only `nets_to_route_`	Add `routing_snapshots_`, `is_incremental_`, `beginIncremental()`
`GridGraph`	No snapshot mechanism	Add `saveSnapshot()`, `restoreSnapshot()`
`GlobalRouter.cpp`	3x `!use_cugr_` bypasses	Conditional CUGR incremental path
`FastRoute.h`	`is_incremental_grt_` never synced	Called from `beginIncremental()`
`CUGR.cpp`	`routeIncremental()` calls full `route()`	New `routeNets(dirty_nets)` function

Target benchmark: `incremental reroute of 10 modified nets on ibex (15,000 nets total) completes in under 5% of full re-route time`

Unforeseen Risks & Mitigation

Correctness Risks

1. **I'm worried about GridGraph edge capacity corruption** after snapshot restore `commitTree()` modifies edge capacities directly, so if restore happens mid-state things can silently break. **I'll add an assert that edge capacity matches the routing tree after every `restoreSnapshot()` call and test this on GCD in Week 2**.
2. **Consecutive `beginIncremental()` calls are a real risk** , the second call clears `nets_to_route_` before the first session finishes, so unprocessed dirty nets get silently dropped. I'll add an assert checking `nets_to_route_.empty()` before clearing and unit test 3 consecutive calls in Week 1.
3. `patternRoute()` might process all nets instead of just dirty ones if `is_incremental_` isn't properly guarding the internal loop. I'll add a guard inside

`patternRoute()` and log the net count processed , it must equal exactly N dirty nets, not the full design count. I'll verify this in Week 3.

Efficiency Risks

1. `routing_snapshots_` can cause a memory spike on 100k+ net designs if the `shared_ptr` map is never cleared after successful routing. I'll call **`routing_snapshots_.clear()`** after every successful `routeNets()` call. Memory overhead for k=10 dirty nets is roughly **10 × 50 nodes × 64 bytes = 32KB** well within the 5% bound. I'll profile on aes in Week 8 using Valgrind to confirm.
2. Overflow net re-addition can create an infinite routing loop if there's no termination condition. I'll add a max 3-iteration counter matching FastRoute's behavior loop when **`(OF_i - OF_{i-1})/OF_{i-1} < 0.05`** or iteration count hits 3. I'll log convergence data on ibex in Week 7.
3. **`std::map has O(log n)`** lookup overhead which adds up at **10,000 dirty nets per** pass. If profiling on aes in Week 8 shows more than 2% overhead from map lookups, I'll swap to `std::unordered_map` for O(1) average case.

Quality of Results Risks

1. **Missing layer assignment is the biggest quality risk.** I confirmed via empty grep that CUGR has zero layer assignment during incremental routing, meaning rerouted nets can get wrong layer assignments and cause **DRC violations**. I'll implement **`layerAssignmentIncremental()`** for dirty nets only in Week 10. Pass criterion is **`check_drc`** showing zero new violations vs full re-route baseline.
2. **NDR clock nets are a timing risk ,there's no `clearNDRnets()` equivalent in CUGR so** nets with non-default routing rules like double-spacing clock nets will be misrouted during incremental **ECO and degrade WNS/TNS**. I'll detect NDR nets before **`addDirtyNet()`** and fall back to FastRoute for those specific nets in Week 11. WNS/TNS must stay within **1% of baseline.**
3. **Incremental routing skips global congestion rebalancing** so it can increase overflow on congested designs. I'll compare overflow count, wirelength, and via count on aes against full re-route baseline. **If overflow increases by more than 5%**, I'll trigger a selective regional re-route for the affected area and document all three metrics as part of the final deliverable.

Availability

I'm available full-time for the entire GSoC period ,May 26 through August 17. **My final year exams at JIS University finish before the Community Bonding period starts, so there's no academic overlap with the coding period.**

I can commit 40+ hours per week during the coding period. My timezone is IST (UTC+5:30) and I'll be online during morning and evening hours.I've already been syncing with Eder on PR reviews across this timezone gap without any issues.

I have no internship, job, or other commitments lined up for this summer. GSoC is my full focus.

I'll be reachable on email and GitHub daily. If anything unexpected comes up I'll inform Eder immediately ,but I'm not anticipating any disruptions.

Future Scope

1. **I will implement timing-driven net prioritization using** `calculatePartialSlack()` to prioritize timing-critical nets during incremental rerouting nets with the worst slack getting rerouted first to maximize WNS improvement per ECO iteration.

2. **I will build a visual congestion heatmap** using `extractCongestionView()` of GridGraph.cpp showing before and after incremental routing comparison ,so engineers can visually identify which regions benefit most from incremental vs full re-route.

3. **I will train a lightweight model on historical ECO** data to predict which nets will cause overflow after incremental rerouting ,pre-emptively selecting better reroute candidates before `addDirtyNet()` is called.

4. **I will parallelize `routeNets(dirty_nets)`** across independent net clusters — nets with no shared edges can be rerouted simultaneously, with an expected 4-8x additional speedup on modern multi-core machines.

5. **I will implement cross-design learning** by sharing routing snapshots across similar designs. If net topology matches a previously seen ECO, I'll reuse the cached routing solution instead of doing a full rip-up and reroute.

6. **I will add `global_route -use_cugr -incremental`** as a first-class option in OpenROAD-flow-scripts ECO repair flow; currently only FastRoute incremental is supported in the automated flow, CUGR incremental has no entry point there.

7. **I will feed CUGR incremental results back to the detailed router** TritonRoute incrementally. Currently TritonRoute reruns on all nets even when only k nets changed globally, which is unnecessary overhead I can eliminate.

About Me

I am a **final-year Computer Science undergraduate** at **JIS University**, Kolkata,India. My **open source journey** started in my very **first semester** when a senior who had contributed to **GSoC** showed me what it meant to build something that actually mattered to people outside your own laptop. **That conversation rewired how I thought about software**.

I did not ease into open source. I started with **Summer of Bitcoin**, contributed to **GirlScript Summer of Code twice**, and then switched sides , **becoming a mentor**. In my 4th semester I was **selected** as an open source mentor for JGEC students, guiding contributors on **DivineShield, a women's safety application** I founded and continue to build.

Teaching others how to contribute taught me more about software quality than any course ever did.

I also built AutoFlow, an RTL2GDS orchestration engine with a C++ DAG scheduler and TCL scripting layer mirroring OpenROAD's pipeline architecture , giving me deep familiarity with the tool chain before I ever opened the codebase.

I am a two-time ICPC Regionalist. **Competitive programming** trained me to read codebases fast, reason about edge cases systematically, and **fix bugs** under pressure ,exactly the skills that showed up in every one of my OpenROAD contributions.

Why I Am the Right Fit

1. **My 3 merged PRs (#9763, #9938, #9870) are all inside CUGR source code , Eder has reviewed my code for weeks and knows my work quality firsthand.**
2. **I found 8 verified bugs with exact file names** and line numbers ,this isn't theoretical knowledge, this is weeks of actual codebase study.
3. **I've already compiled OpenROAD from source, run the full nangate45/gcd pipeline, and verified every bug I documented actually exists.**
4. **ICPC Regionalist 2024 and 2025 , I can read complex C++ codebases fast, reason about edge cases, and debug under pressure.**

5. I built **AutoFlow, an RTL2GDS** orchestration engine with a C++ DAG scheduler and TCL scripting proving I understand OpenROAD's architecture from first principles.

Why I Chose This Project

I chose this project because I found the bugs before I knew there was a GSoC slot for it.I was studying CUGR's incremental path to understand how OpenROAD handles ECO workflows. I expected to find a working implementation with maybe a few gaps. Instead I found `endIncremental() setting is_incremental_ = true` instead of **`false`** meaning the router never actually exits incremental mode. I found `routeIncremental()` calling full `route()` internally , rerouting all 50,000 nets when only 3 need rerouting. I found dirty nets being tracked in `nets_to_route_` but completely ignored because `route()` builds its own net indices from scratch and never reads it.

These aren't minor gaps. These are fundamental bugs that make CUGR's incremental routing completely non-functional. Every timing engineer using CUGR is hitting these bugs right now without knowing why.

I didn't pick this from a list. I opened the codebase, couldn't stop reading, and by the time I **understood exactly what was broken I already knew exactly how to fix it. That's the only reason I'm here.**